MiniCDN SRS, UML and algorithm design

Group 21, CE, AY 2026-27. Guide: Dr. Sudhir N. Dhage. Revision: 10 October 2026.

Purpose and problem

Build an inspectable distributed static-content delivery prototype. Repeated origin fetches waste work, while routing every popular key to its hash owner can concentrate requests. Spreading every request uniformly can duplicate cached objects and reduce effective cache capacity. The project exposes that tradeoff through controlled algorithms, observable cache behavior, and reproducible workloads.

The project contribution is an experimental platform and an evaluated routing heuristic. Consistent hashing, LRU, conditional HTTP validation, request coalescing and cache-aware balancing are established techniques. No claim that commercial CDNs lack them is justified by this implementation.

Stakeholders and operating assumptions

Content clients request public static assets. Operators start services and issue invalidation requests. Evaluators inspect code, tests, data and Grafana. The origin is the authoritative content source. Redis is the single control-plane registry and invalidation-generation store. All three edges in the local demonstration share one host/network; region labels do not prove geographic distribution.

Edge cache identity is the decoded request path plus raw query, within one configured origin. The router hashes the escaped request URI including its raw query. Query ordering is preserved; equivalent encoded spellings may route differently. Encoded-path normalization requires additional validation before wider deployment. Authorization, Cookie, Range and explicit request Cache-Control bypass shared storage. Vary and Set-Cookie responses are excluded. GET responses with status 200 and positive permitted TTL may be cached; HEAD can use a cached GET body's headers without returning the body. The implementation is a scoped prototype, not complete RFC compliance or multi-tenant CDN hosting.

Functional requirements and evidence

ID	Requirement	Acceptance/evidence
FR01	Serve origin content through router and edge	HTTP 200 body matches the requested fixture; query variants remain isolated
FR02	Provide selectable round robin, hash and adaptive policies	ROUTING_MODE validated at startup; response identifies mode; selection tests and common workload traces
FR03	Enforce memory-body capacity and LRU eviction	CACHE_MAX_BYTES, oversized rejection, replacement accounting and least-recent eviction tests; metadata is extra; entry count capped at 10,000
FR04	Coordinate concurrent same-key cacheable fills	32-client controlled burst compares independent origin attempts to one coordinated fill; independent keys continue
FR05	Respect freshness and conditionally validate stale entries	Expired content is withheld until origin validates or replaces it; 304 renews body freshness; private/no-store bypass storage
FR06	Invalidate exact keys and URL prefixes	POST purge acknowledged per active edge; prefix includes matching query variants; partial failures return HTTP 502
FR07	Recover invalidations missed by returning edges	Redis increments generation before broadcast; a detected gap causes a conservative full-cache flush; heartbeat checks retained generation
FR08	Discover healthy edges and fall back	One-second discovery loop, parallel one-second health probes; no locks held across proxy I/O; failed GET/HEAD connection attempts can try another edge
FR09	Export operating metrics and dashboard	Content traffic, hits, latency, origin attempts, coalesced requests, body bytes, evictions and healthy edge count visible through Prometheus/Grafana
FR10	Reproduce experiments	Seed, capacity, workload, concurrency, origin delay, all errors and raw request timings recorded; cold state reset acknowledged by all edges

Nonfunctional requirements

NFR01 Correctness: no stale object is served merely because it remains in the bounded cache; validation is mandatory after expiry. Client conditional headers take precedence by ETag before modification date. Oversized bodies stream without admission. Coalescing shares only cacheable content.

NFR02 Concurrency: shared state is locked; fills are keyed; cancelled waiting requests release references. Purge uses an exclusive edge cache-operation barrier. This favors correctness but can make purge wait for slow responses, a measurable limitation.

NFR03 Performance: measure P50/P95/P99, successful throughput, request-count coefficient of variation, cache hit ratio and origin attempts. The proposal's >85% hit and <5 second failover are workload-dependent targets, not blanket guarantees. Router connections have a 500 ms dial deadline and 1.5 s response-header deadline after a measured eight-second failure exposed a problem. The complete response retains a 12 s

timeout, so a stalled body or host scheduling delays can still exceed the target. Record the tested failure type and timing boundary.

NFR04 Reproducibility: closed-loop clients, equal trace/capacity/TTL and sequential policy trials. Repeat three times and rotate policy order. Record machine/container versions; report ranges across runs. Prometheus histogram percentiles are estimates and do not replace raw client latency samples.

NFR05 Deployment: expose only HTTPS entry points on a VPS. Keep Redis, control, edges and Prometheus internal. Require a non-default Grafana password. Persist Redis AOF and Grafana data. Public hosting requires a host, DNS and credential configuration.

NFR06 Maintainability: separate origin, control, router, cache, hash and edge modules. Keep tests beside modules, a versioned experiment script and concise commit/branch conventions. A clean passing revision is required before release.

Interface contract

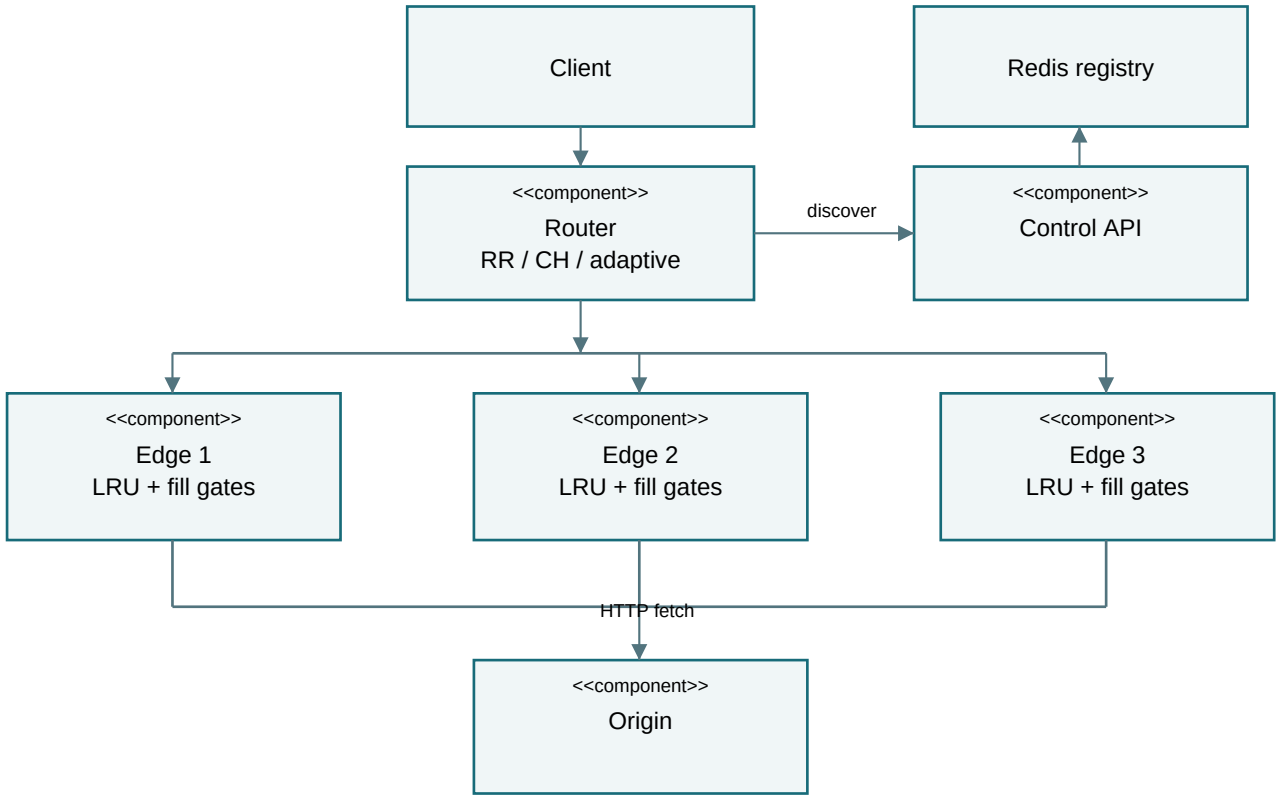
GET/HEAD /{resource}: client content path via router. X-Edge-Node identifies serving edge, X-Cache reports HIT/MISS/REVALIDATED/BYPASS, X-Routing-Mode identifies policy.

GET /api/v1/nodes: control returns currently leased nodes. POST /api/v1/nodes: heartbeat registration, returns purge_generation. Five-second heartbeat, 15-second lease. Router independently probes /health.

POST /api/v1/purge: {"urls":["/file?x=1"],"prefixes":["/assets/"]}. Control persists generation and broadcasts /internal/purge. Empty URL/prefix arrays are rejected. Prefixes must begin with /. The local control API is unauthenticated and must remain internal for public hosting.

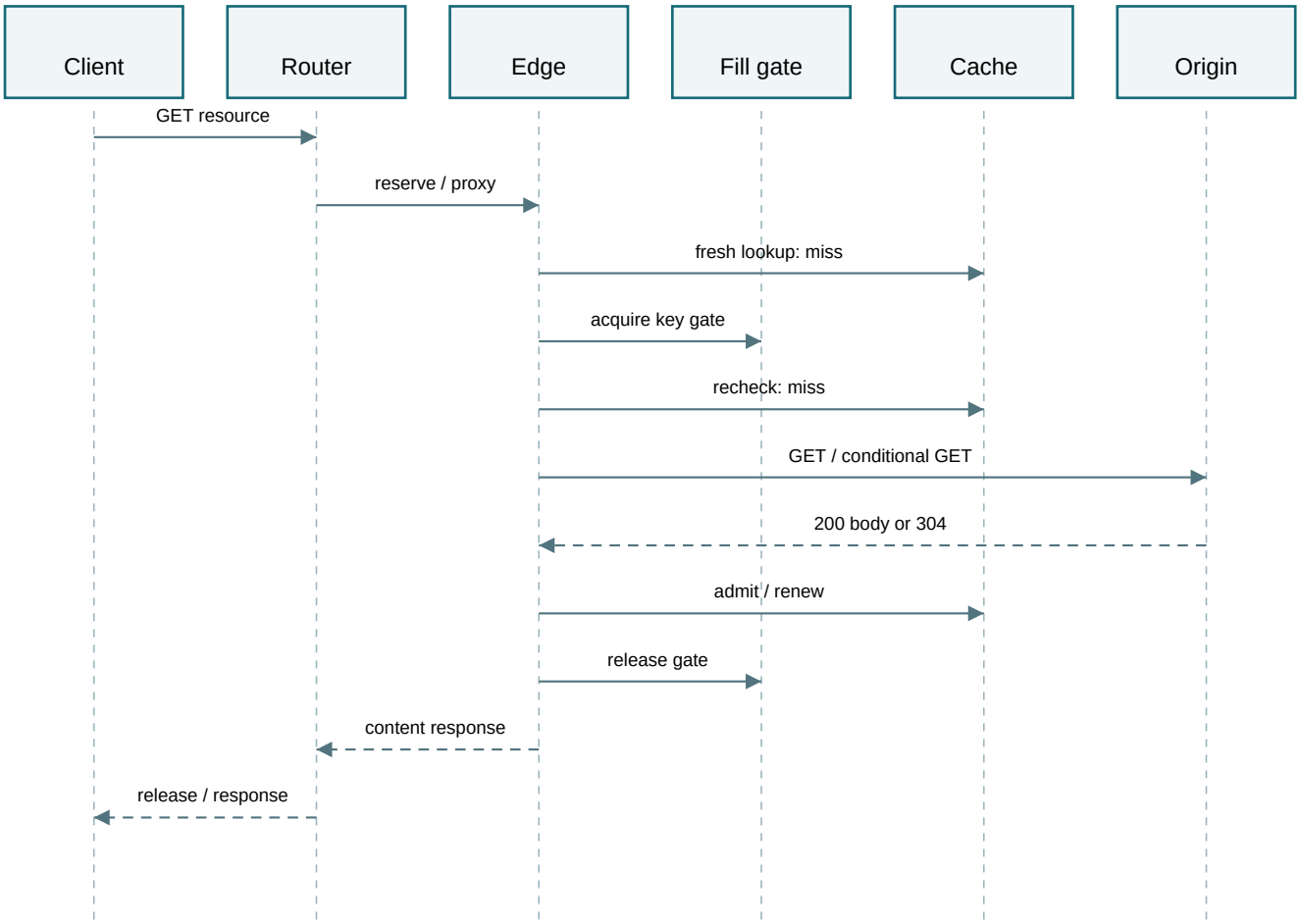
GET /metrics: edge/router Prometheus scrape interface. GET /health: process health and operating summary. Registry membership is not a substitute for a successful health check.

UML source: component/deployment



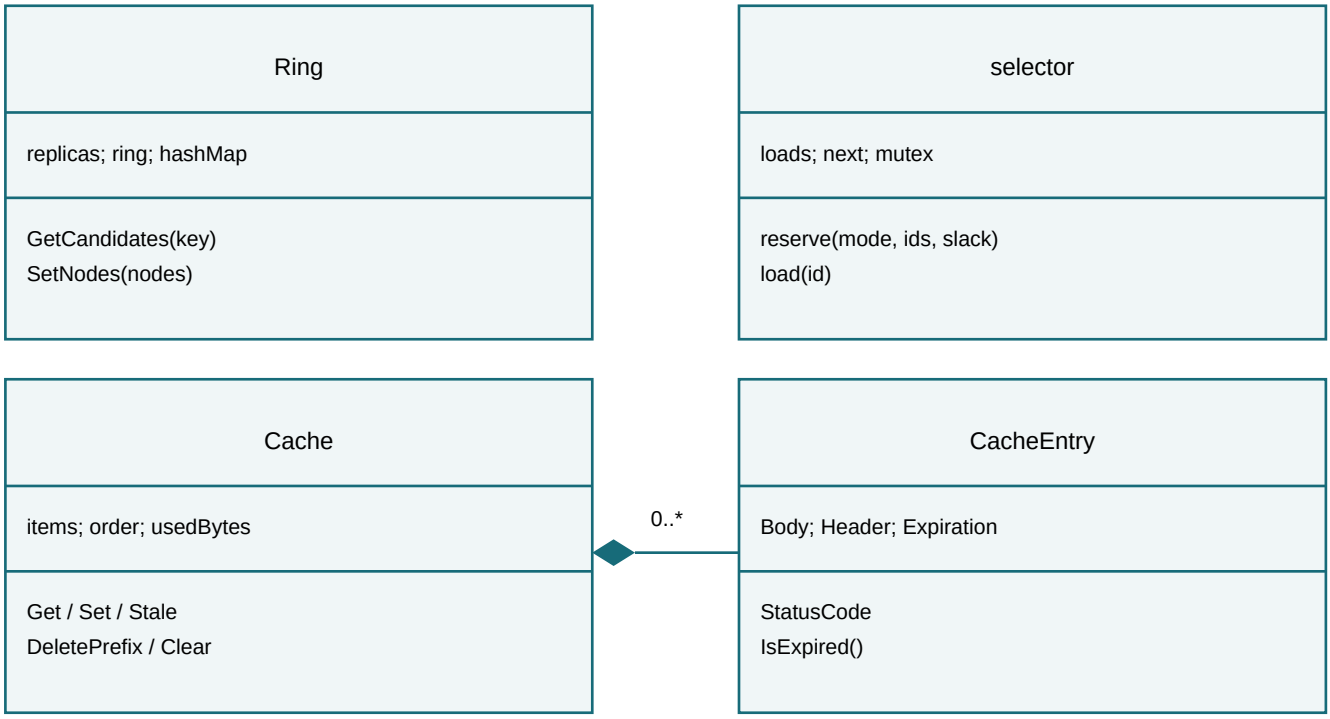
Prometheus scrapes router/edges; Grafana queries Prometheus.

UML source: cache miss sequence



Cacheable GET miss path. Separate UML source contains alternative branches.

UML source: class relationships



Algorithm design

Round robin: snapshot healthy IDs, sort lexically, choose next modulo N under selector mutex, increment counter and reserve one in-flight request. Return remaining IDs in cyclic fallback order. Complexity O(N log N) for the small snapshot implementation. Equal request counts do not imply equal processing cost.

Consistent hashing: hash full path/query with CRC32, binary-search 50 virtual nodes per physical edge, traverse clockwise for unique fallback candidates, and reserve the first healthy candidate. Owner lookup is O(log V); complete candidate generation can scan O(V), with V=50N. This CRC32 ring is a demonstrator and its empirical distribution must be measured rather than assumed uniform.

Adaptive spillover: begin with hash owner and find the least-loaded healthy candidate using router-local in-flight counts. Keep owner unless owner_load > minimum_load + ADAPTIVE_SLACK (default 2). Otherwise use least-loaded candidate. Reserve atomically with selection, release on completion, account for fallback attempts separately. The rule has O(N) load selection after candidate construction. Multiple routers do not share load state, so experiments use one active policy router at a time.

Fill coordination: perform initial cache lookup, acquire a cancellable per-key gate on GET miss, recheck cache without double-counting lookup, fetch/validate only if needed, publish eligible response and release gate before serving body. Followers reuse the completed fill. Large/noncacheable responses are not admitted or shared as cached responses.

Revalidation: retain expired entry under the same capacity bound, send its ETag or Last-Modified validator, merge 304 headers, renew permitted TTL and serve retained body. A 200 response replaces content only if cacheable. No validation means no stale serving.

Invalidation: persist incremented generation in Redis, broadcast exact/prefix purge, return per-edge acknowledgements. An edge applies purge behind an exclusive cache-operation barrier. A generation gap forces full flush; a later heartbeat with newer generation also forces full flush. This is centralized recovery, not gossip, globally atomic invalidation, or a guarantee of zero stale reads while disconnected.

Test traceability and release limits

FR02: cmd/router/routing_test.go. FR03: pkg/cache/bounded_test.go. FR04: cmd/edge/coalescing_test.go. FR05: cmd/edge/main_test.go and revalidation_test.go. FR06/07: purge_recovery_test.go and live interrupted-node test. FR01/08/09: live Compose smoke/failure/dashboard checks. FR10: scripts/compare-routing.mjs plus recorded JSON/CSV/raw samples.

Disk caching remains absent. Cache body bytes exclude metadata, temporary response buffers and Go runtime overhead. Automatic cross-edge replication and global origin-budget enforcement remain future work. HTTP upstream is local/trusted. A VPS deployment remains a single host with multiple processes and does not demonstrate geographical CDN performance.

Integration and experimental evidence

Measurements come from the recorded local Docker trials. Means below average repetitions within each workload/policy; they do not combine unlike workloads. See the CSV and JSONL for individual trials and request timings.

Workload / policy	Hit %	Origin attempts	P95 mean [range] ms	Errors
uniform / consistent_hash	60.9	300.3	235.4 [216.4-259.8]	0
uniform / round_robin	21.1	606.0	314.6 [217.1-455.3]	0
uniform / adaptive	48.4	396.0	232.0 [129.7-369.0]	0
zipf / consistent_hash	80.3	151.0	130.3 [88.1-210.5]	0
zipf / round_robin	65.5	265.0	146.0 [104.0-210.2]	0
zipf / adaptive	76.7	179.0	161.8 [99.3-264.3]	0
changing_hot / consistent_hash	77.0	176.3	198.9 [164.6-232.8]	0
changing_hot / round_robin	59.3	312.7	243.2 [146.9-400.4]	0
changing_hot / adaptive	71.5	218.7	265.1 [132.3-441.1]	0
cold_burst / consistent_hash	99.9	1.0	118.2 [65.5-165.2]	0
cold_burst / round_robin	99.6	3.0	114.0 [56.0-229.9]	0
cold_burst / adaptive	99.6	3.0	195.5 [66.7-449.0]	0

This comparison uses three edges on one machine, 64 KiB body capacity per edge, 256 one-KiB objects, concurrency 16, 768 requests per trial and 20 ms synthetic origin delay. All cache and origin policy settings are shared. Live Grafana aggregates traffic across trials; the trial counters and client trace are the comparison evidence.

36 trials completed on 10 October 2026: 27,648 requests and zero request/body-validation errors. Seed 2100260; three repetitions; cold caches and no warmup; policy order rotates. P95 values are means of trial percentiles, with minimum and maximum across runs, not pooled request percentiles or confidence intervals.

Consistent hashing reduced origin work relative to round robin in each tested workload. Round robin had exactly equal request counts on the three healthy edges. Adaptive routing traded some locality for request balance and did not consistently improve P95. The wide latency ranges indicate host variability; these results do not establish a production or regional advantage. Uniform-cache hits were below the proposal target of 85%, so that target has not been generally achieved.

Functional integration checks

Check	Result
Three-edge purge acknowledgement	PASS
Router-origin body fidelity and MISS to HIT	PASS
HEAD uses cached headers without body	PASS
Cached ETag conditional GET	PASS
Personalized requests bypass shared cache	PASS
Missing content remains HTTP 404	PASS
Locate edge-1 hash owner before failure	PASS
Hard-stopped edge leaves content available	PASS
Post-stop request completes within five-second target	PASS
Unavailable edge purge reports partial failure	PASS
Returning edge retrieves current content	PASS
Disconnected edge purge reports failure	PASS
Missed invalidation recovered without process restart	PASS
Partition recovery preserves edge process	PASS
Grafana database healthy	PASS
Prometheus targets recovered	PASS

One request after its owning edge was stopped completed in 567 ms. Timing begins after docker stop completes; it is not a distribution of crash detection delays. A separate network-disconnection test verified missed-purge recovery without a process restart. Native Go 1.27 tests and Linux Go 1.21 race tests passed; the coordinated cold-burst test was repeated ten times.

MiniCDN delivery plan and evaluation evidence

Updated 10 October 2026. Dates below were supplied by Jaynish in chat; the attached marksheets contain blank date fields. The marksheets are evaluation context, not instructions to fabricate grades, signatures, meetings, publication acceptance, or completed results.

Review	Deadline	Evaluation	Evidence to prepare
Phase I	17 August 2026	Guide	Problem statement, related work, planning, presentation
Phase II	2 September 2026	Guide	SRS (10 marks), UML (5), algorithm design (5), presentation (5)
Phase III	30 September 2026	Guide	Module coding (10), Git setup (5), branching/commit rules (5), presentation (5)
Phase IV	12 October 2026	Guide	Integration (10), functional/nonfunctional tests (5), report (5), presentation (5)
ESE	2 November 2026	Industry expert/alumni	Definition (20), presentation/demo (20), work done (30), Q&A (20), poster/research paper (10)

The first three dates have passed. That does not establish whether those evaluations/submissions happened. Phase IV is the immediate priority. ESE passing mark in the supplied sheet is 40/100.

Submission scope

Deliver a reproducible static-content CDN prototype with origin, Redis control plane, three Go edges, three routing policies, bounded memory LRU, TTL and conditional validation, duplicate-fill suppression, targeted/prefix invalidation, reconnect invalidation recovery, and Grafana/Prometheus. Compare policies under identical seeded workloads. Treat measurable improvements as hypotheses until benchmark data exists.

The initial proposal also mentions a disk tier. It remains an implementation gap and must not be described as complete. HTTP/3, Wasm, gossip, predictive prefetch, peer caching, dynamic virtual nodes, regional placement, and switching LRU/LFU are separate extensions, not mandatory implications of the phrase intelligent caching. Confirm any guide-required extension before changing the declared scope.

10-12 October: Phase IV recovery schedule

These are proposed work slots, not invented historical meetings or official extra deadlines.

1. 10 October: repair Docker, finish core policies/cache correctness, collect five papers, finish SRS/UML/algorithm descriptions and deployment configuration.

2. 11 October: run functional integration and repeated workload comparison, exercise node failure and missed purge recovery, capture Grafana evidence, finish report and presentation.

3. 12 October: freeze tested revision, rehearse a short repeatable demo, verify submitted artifacts and guide's required format. Use local demo if public hosting is not ready.

13 October-2 November: ESE

Expand the literature survey from five deeply reviewed papers to at least 12, as required by the supplied Overleaf template. Add recent work and explicit comparison assumptions. Complete the 25-40 page report, architecture/UML, measured results, limitations, budget, references, actual plagiarism report, and poster/research-paper material. The template's sample electrical references are unrelated placeholders and must be replaced. Certificates, signatures, acceptance evidence and plagiarism results require real human/institutional records.

Suggested sequence: 13-18 October additional literature and any approved disk tier; 19-24 October broader validation and sensitivity study; 25-29 October report/poster/public dashboard; 30 October-1 November viva practice and backup demo. These are proposed team milestones.

Team records

Group 21: Jaynish Thakar (2024300260), Anshul Warade (2023300262), Parth Thakkar (2025301033). Faculty guide: Dr. Sudhir N. Dhage. Names/UCIDs come from the supplied group PDF.

Suggested responsibility split for team approval: routing/experiments; cache/control/testing; report/dashboard/presentation. Record the actual owner, commits and review notes; do not assign fictional contributions retrospectively. Weekly form should use real meeting dates and actual supervisor feedback, with signatures left for the signatories.

Completion ledger

- [x] Extract/read supplied rubrics and group proposal; record corrected deadlines.

- [x] Implement round robin, consistent hashing and request-load-aware spillover.

- [x] Add bounded response-body LRU, object streaming limit, expired-object validation, prefix purge.

- [x] Persist purge generation in Redis and flush missed invalidations on heartbeat (implementation).

- [x] Verify generation recovery and concurrency with tests and live integration (16 checks passed).

- [x] Complete Docker startup recovery, container build, compatibility/race tests.

- [x] Collect full five-paper sources and complete cited research report.

- [x] Run comparison and node-failure experiments; save raw traces and environment metadata (36 trials, 27,648 requests, zero request errors).

- [x] Finish Phase IV report, UML, presentation and Grafana screenshots using actual results.

- [ ] Obtain hosting account/server, connect metrics.jaynishthakar.tech, verify HTTPS/authentication.

- [ ] Final ESE report: 12 papers minimum, 25-40 pages, poster/research paper, plagiarism proof and signatures.